

PROJECT REPORT

Personal Password Strength Auditor

An Offline Java Tool for Local Password Strength Analysis, Pattern Detection and Entropy Estimation

Course Title: Programming in Java

Course Code: CSE2006

Academic Year: 2026-27

Submitted by: Vinayak Vikram Singh

Roll Number: 25BAI10262

Institution Name: VIT University Bhopal

1. Introduction

Passwords remain the form of authentication on the internet yet passwords also are the weakest link. I see how passwords can be weak. Studies of leaked databases consistently show that a small set of passwords – things like "password", "123456", "qwerty" and pet names – account for a share of what real people actually use. When a website is breached attackers do not try every string; they try the likely ones first and they succeed far more often than they should.

The Personal Password Strength Auditor is a command-line tool written in Java that reads a list of passwords from a file and tells the user, for each password how strong it really is. For every password it reports the length the character classes used any patterns it contains (passwords, keyboard walks, sequences, years dictionary words) an entropy estimate and an approximate crack-time. I appreciate that the entire analysis happens offline – no password ever leaves the user's machine. Crucially the tool never sends your passwords to the internet.

This project was built as a learning exercise covering string analysis, regular expressions, elementary information-theory entropy and good security hygiene. I built this project to learn about these topics. It is organized into modules with no external dependencies, beyond the Java standard library and JUnit for testing.

The report that follows documents the problem the requirements, the architecture, the design decisions and the results.

2. Problem Statement

2.1 The Problem

Most people reuse a handful of weak passwords across many accounts, and the passwords they choose tend to be predictable. Users have almost no way to see how weak a password really is: online strength meters are usually just a coloured bar with no explanation, and typing a real password into a random website is itself a security risk. There is a need for a tool that audits a batch of passwords locally, with full transparency about what it found and why a password scored the way it did.

2.2 Objectives

- Analyse passwords purely locally.
- Give actionable, human-readable feedback so users learn why a password is weak.
- Combine mathematical entropy estimation with pattern-based detection.
- Keep the codebase modular and testable, with each concern in its own module.
- Mask passwords in all output so reports are safe to screenshot and share.

2.3 Scope

The tool is a fully offline command-line application. Given a local text file containing one password per line, it produces a per-password report and a summary. It deliberately does not store, transmit, or persist any password. The only side effects are console output and an optional self-contained HTML file. It uses no external libraries and no database.

2.4 Target Users

- Everyday users who want to check passwords before adopting a password manager, without trusting a third party.
- Students and educators using it as a teaching aid for strings, regex, entropy and security hygiene.
- Developers wanting a lightweight, dependency-free reference for how strength estimation works.

3. Functional Requirements

The system provides the following functional capabilities, each backed by a dedicated module:

ID	Requirement	Module
FR1	Read passwords from a local file (one per line)	input.PasswordFileReader
FR2	Measure password length and character-class diversity (a-z, A-Z, 0-9, symbols)	analyzer.CharacterClassAnalyzer
FR3	Detect weak patterns: common/leaked passwords, repeating chars, sequential digits/letters, keyboard walks, years, dictionary words	analyzer.PatternDetector
FR4	Estimate entropy in bits using the simplified Shannon formula	entropy.EntropyCalculator
FR5	Estimate crack-time and assign a strength verdict (Very Weak to Very Strong)	entropy.CrackTimeEstimator, model.PasswordStrength
FR6	Orchestrate the full analysis pipeline per password	analyzer.PasswordAnalyzer
FR7	Render a readable console report with per-password detail and a summary	reporting.ConsoleReporter
FR8	Render a self-contained HTML report with masked passwords	reporting.HtmlReporter
FR9	Provide a CLI entry point with argument parsing and error handling	Main
FR10	Mask all passwords in output so reports are safe to share	reporting.ConsoleReporter / HtmlReporter

4. Non-Functional Requirements

Attribute	Description
NFR1 Security / Privacy	All processing is local. No network sockets are opened, no passwords are written to disk beyond the optional masked HTML report, and no telemetry is collected. Passwords are masked in all output. Input lines longer than 1024 characters are rejected to guard against binary-file ingestion.
NFR2 Performance	Each password is analysed in microseconds; a batch of thousands completes well under a second. Pattern detection uses compiled regexes and a HashSet lookup, giving $O(n)$ per password where n is its length. No blocking I/O occurs after the initial file read.
NFR3 Usability	The CLI has a single clear usage string and sensible defaults. Output is aligned and colour-coded (HTML), with actionable warnings in plain English. The console and HTML reports are self-documenting.
NFR4 Reliability & Error Handling	Missing files, null passwords, empty inputs and malformed lines are handled gracefully with informative messages and non-zero exit codes. Null passwords are treated as empty rather than crashing.
NFR5 Maintainability	Each concern lives in its own package and class. Constants are centralised in Config. Report objects are immutable. The code is commented throughout and ships with unit tests for every module.
NFR6 Portability	Pure Java 17 standard library - no native or platform-specific code. Runs on any OS with a JVM. The only build requirement is Maven 3.6+.

5. System Architecture

The application follows a layered architecture with four layers. Data flows top-down: the user provides a password file, the input layer reads and validates it, the logic layer performs all analysis, and the presentation layer formats the results. No layer depends on the one above it, which keeps the design clean and testable.

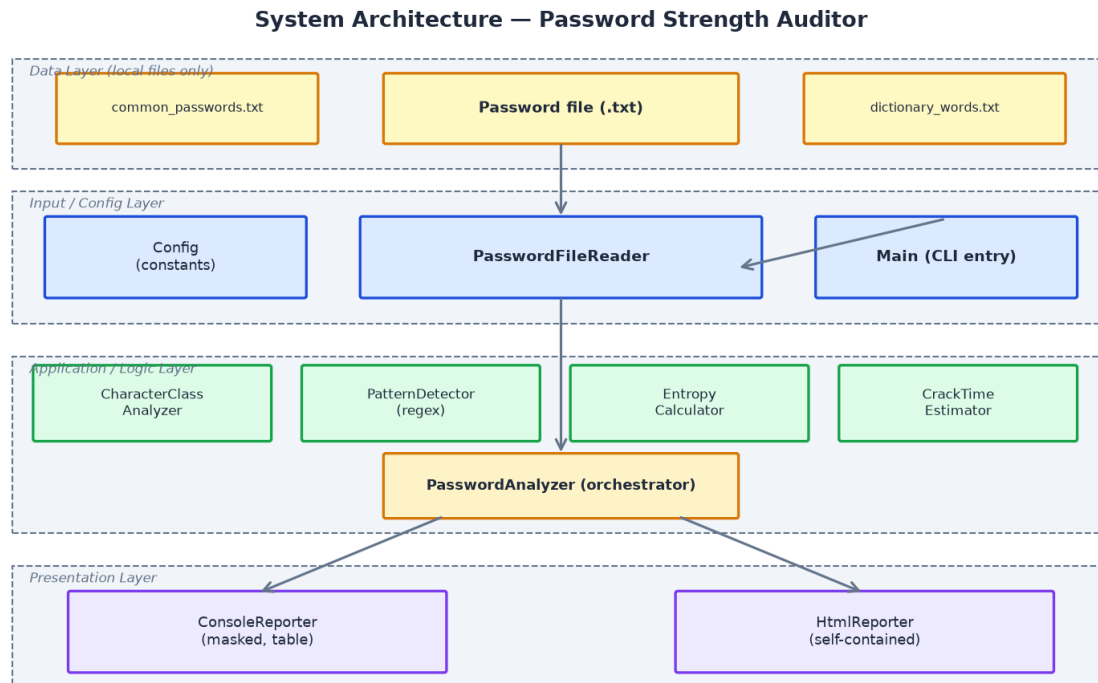


Figure 1. Layered system architecture

The Data Layer holds only local text files: the password file under audit and two reference word lists (common passwords and dictionary words). The Input/Config Layer contains the file reader, the centralised Config, and the Main entry point. The Application Layer is where the real work happens - four independent analysers coordinated by a single PasswordAnalyzer orchestrator. The Presentation Layer offers two interchangeable reporters (console and HTML) that consume the same immutable report objects.

6. Design Diagrams

6.1 Use Case Diagram

The user interacts with the system through five primary use cases, all of which execute locally.

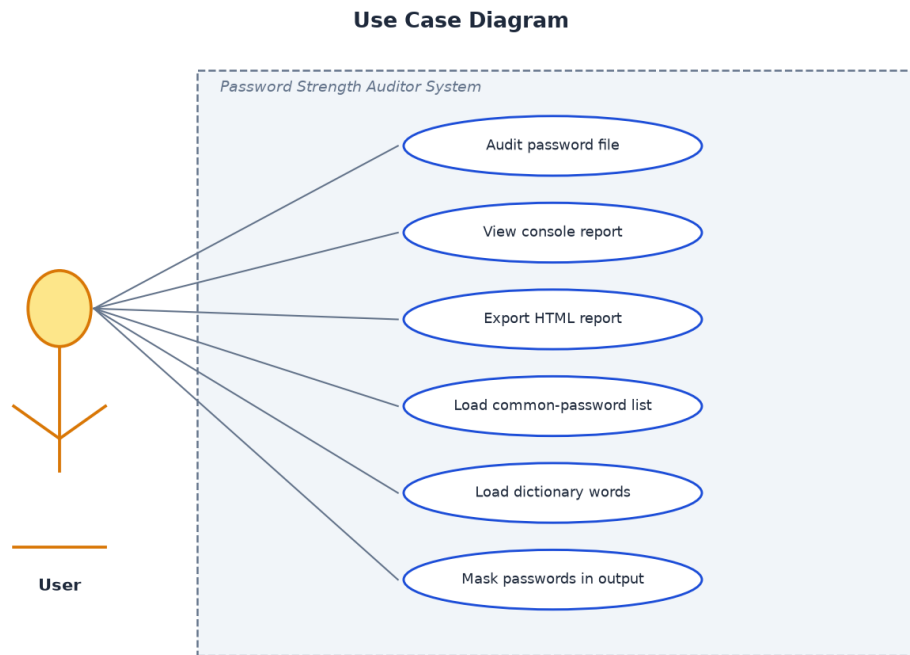
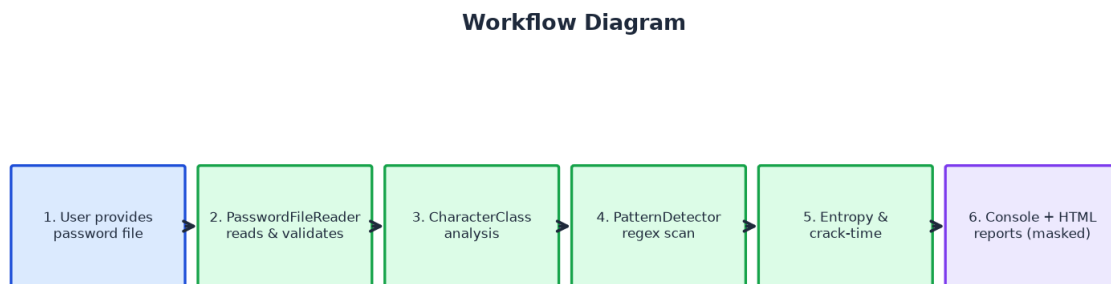


Figure 2. Use case diagram

6.2 Workflow Diagram

The end-to-end workflow proceeds in six stages, from file input to masked reports.



All processing is local — no network access at any step

Figure 3. Workflow diagram

6.3 Sequence Diagram

The sequence below shows the message flow for auditing an entire password file, from the user's command to the final output.

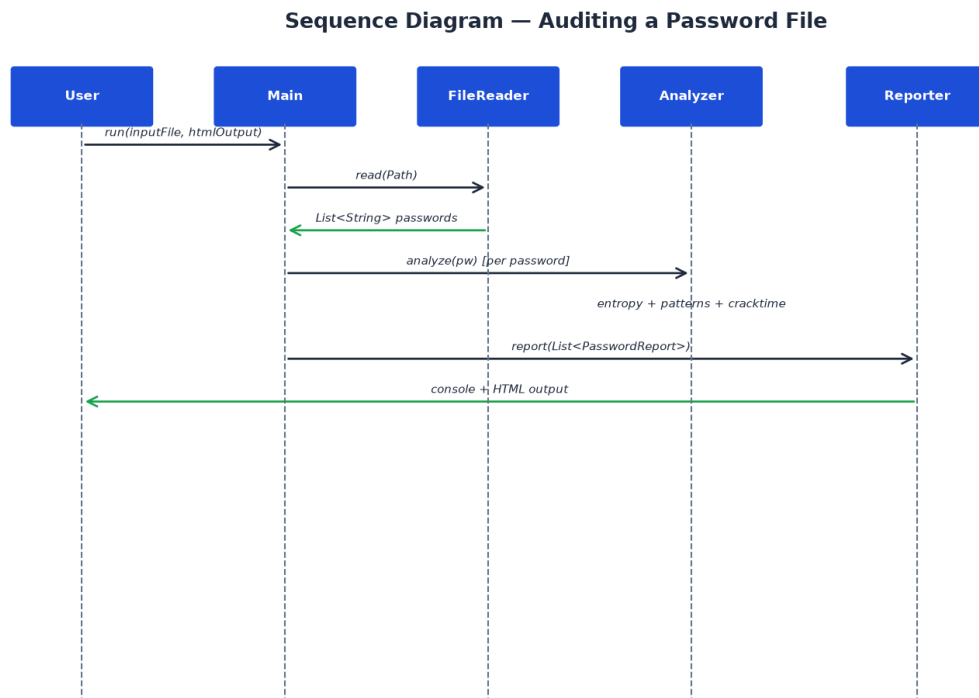


Figure 4. Sequence diagram

6.4 Class / Component Diagram

The class diagram shows the core types, their responsibilities, and the key dependencies between them.

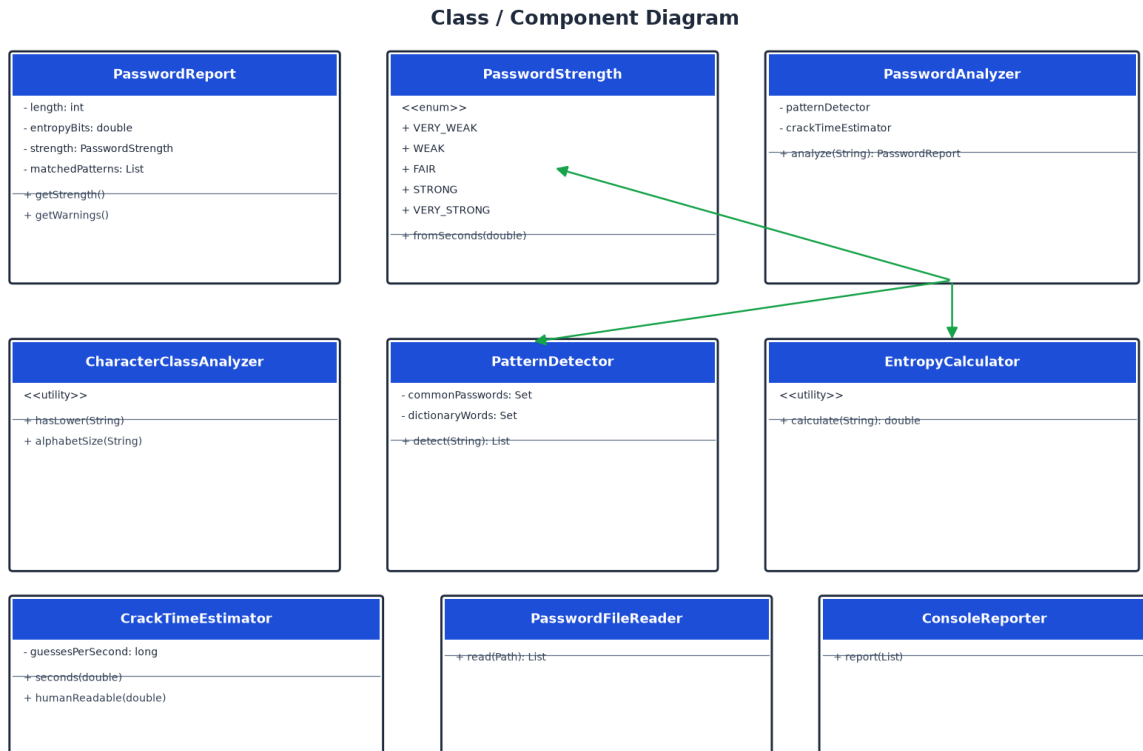


Figure 5. Class / component diagram

6.5 Data Design

The project does not use a relational database, so there is no ER diagram. Data is stored as plain text files read on demand. The three files and their schemas are:

File	Format	Purpose
common_passwords.txt	One lowercase password per line	Used for exact and substring matching
dictionary_words.txt	One dictionary word per line	Flagged when found inside a password
sample_passwords.txt	One password per line (the input)	The list to be audited

7. Design Decisions & Rationale

7.1 Why Java and the standard library only

Java was chosen because the project maps naturally onto object-oriented design and because the standard library provides everything needed - NIO for file reading, regex via `java.util.regex`, and streams for character analysis. Avoiding external libraries keeps the build simple and the dependency surface at zero, which matters for a security-sensitive tool.

7.2 Why a layered, package-per-concern structure

Each concern (input, analysis, entropy, reporting, model) lives in its own package. This makes the code easy to navigate, lets each package be tested in isolation, and means a new report format or a new pattern detector can be added without touching existing code - the open/closed principle in practice.

7.3 Why combine entropy with pattern detection

Entropy alone is misleading: "aaaaaaaaaa" has non-trivial raw entropy but is trivially crackable. Patterns alone are also incomplete: a long random string with no patterns is genuinely strong. The tool runs both and surfaces both, so the user gets the full picture. This is the single most important design decision in the project.

7.4 Why immutable report objects

PasswordReport is built via a Builder and is immutable once created. This guarantees that a report cannot be accidentally modified after analysis, which is valuable when the same report is consumed by multiple reporters. It also makes the objects trivially thread-safe.

7.5 Why mask passwords in output

Even though the tool is offline, its console and HTML output may be screenshotted or shared. Masking (showing only the first and last character) lets the user recognise which password is which without the full secret appearing in a report they might forward to someone else.

7.6 Why a simplified entropy formula

A full zxcvbn-style probabilistic model was out of scope. The textbook formula $\text{entropy} = \text{length} \times \log_2(\text{alphabetSize})$ is transparent, easy to test, and easy to explain to a reader - which suits the educational goal. The pattern detector compensates for the formula's blindness to predictability.

8. Implementation Details

8.1 Module overview

Module	Responsibility
Config	Centralises tunable constants: attack rate, minimum length, file paths.
PasswordReport / PasswordStrength	Immutable report DTO with a Builder; enum mapping crack-time to a strength bucket.
CharacterClassAnalyzer	Detects which of the four character classes are present and computes the alphabet size.
PatternDetector	Runs regex-based checks for repeats, sequences, keyboard walks, years, plus dictionary/common-password lookups.
EntropyCalculator	Applies $\text{entropy} = \text{length} \times \log_2(\text{alphabetSize})$.
CrackTimeEstimator	Converts entropy to seconds using $2^{(\text{entropy}-1)}/\text{rate}$, then formats it for humans.
PasswordAnalyzer	Orchestrates the pipeline: classes -> patterns -> entropy -> crack-time -> report.
PasswordFileReader	Reads and sanitises the input file, rejecting overlong lines and blank entries.
ConsoleReporter	Prints an aligned per-password table and a summary to the terminal, with masking.
HtmlReporter	Writes a self-contained, styled HTML page with a masked results table.
Main	Parses CLI arguments, loads word lists, runs the pipeline, dispatches to reporters.

8.2 The entropy and crack-time model

For a password of length L drawing from an alphabet of size A , the entropy in bits is:

$$\text{entropy} = L \times \log_2(A)$$

where A is the sum of the classes present (lowercase 26, uppercase 26, digits 10, symbols 33). The average time to crack is:

$$\text{crackTime} = 2^{(\text{entropy} - 1)} / \text{guessesPerSecond}$$

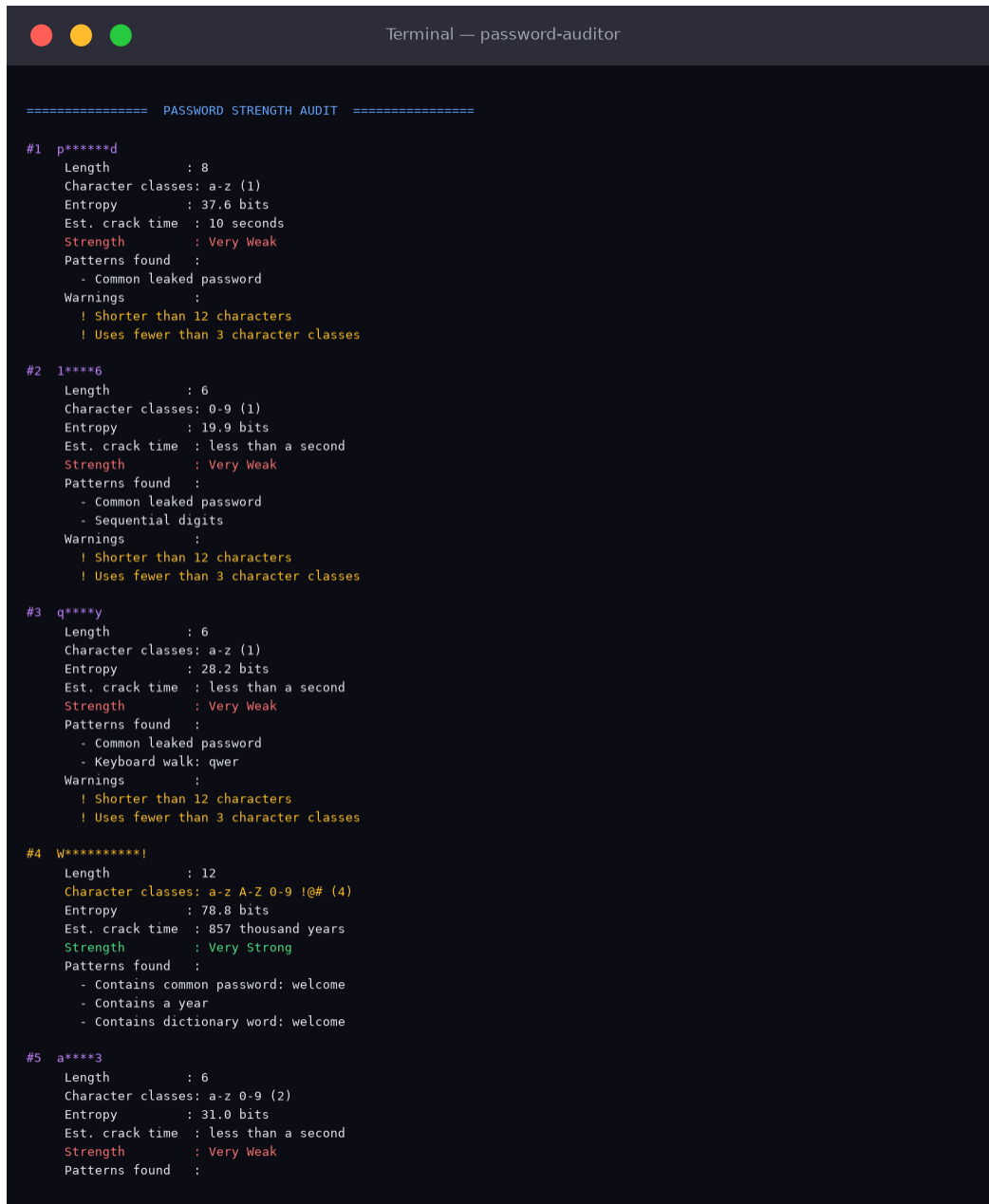
using an offline attack rate of 10 billion guesses per second. The -1 accounts for finding the password on average halfway through the keyspace. This is a simplified textbook estimate; the pattern detector compensates for its blindness to predictable structure.

8.3 Pattern detection techniques

- Common/leaked passwords: exact match and substring containment against a HashSet-loaded list.
- Repeating characters: regex `(.)\1{2,}` catches 3+ identical characters in a row.
- Sequential digits/letters: fixed regex alternations for forward and backward runs of 4.
- Keyboard walks: sliding window of 4 keys along each QWERTY row, checked both directions.
- Years: regex `(19|20)\d{2}` catches 1900-2099.
- Dictionary words: substring containment against a small word list (length ≥ 4 to limit false positives).

9. Screenshots and Results

The tool was run against the bundled sample password list. The screenshots below show the console output and the generated HTML report.



```
Terminal — password-auditor

===== PASSWORD STRENGTH AUDIT =====

#1 p*****
Length      : 8
Character classes: a-z (1)
Entropy     : 37.6 bits
Est. crack time : 10 seconds
Strength    : Very Weak
Patterns found :
- Common leaked password
Warnings     :
! Shorter than 12 characters
! Uses fewer than 3 character classes

#2 1****6
Length      : 6
Character classes: 0-9 (1)
Entropy     : 19.9 bits
Est. crack time : less than a second
Strength    : Very Weak
Patterns found :
- Common leaked password
- Sequential digits
Warnings     :
! Shorter than 12 characters
! Uses fewer than 3 character classes

#3 q****y
Length      : 6
Character classes: a-z (1)
Entropy     : 28.2 bits
Est. crack time : less than a second
Strength    : Very Weak
Patterns found :
- Common leaked password
- Keyboard walk: qwerty
Warnings     :
! Shorter than 12 characters
! Uses fewer than 3 character classes

#4 W*****!
Length      : 12
Character classes: a-z A-Z 0-9 !@# (4)
Entropy     : 78.8 bits
Est. crack time : 857 thousand years
Strength    : Very Strong
Patterns found :
- Contains common password: welcome
- Contains a year
- Contains dictionary word: welcome

#5 a****3
Length      : 6
Character classes: a-z 0-9 (2)
Entropy     : 31.0 bits
Est. crack time : less than a second
Strength    : Very Weak
Patterns found :
```

Figure 6. Console output for the sample password batch

Password Strength Audit						
Generated locally. No password data left your machine.						
#	Password	Len	Cls	Entropy	Crack time	Strength
1	p*****d	8	1	37.6	10 seconds	Very Weak
2	1****6	6	1	19.9	< 1 second	Very Weak
3	q****y	6	1	28.2	< 1 second	Very Weak
4	W*****!	12	4	78.8	857K years	Very Strong
5	a****3	6	2	31.0	< 1 second	Very Weak
7	P*****d	8	4	52.2	1.0 days	Fair
11	7\$Kp#vL2!zQwR	13	4	84.7	9M years	Very Strong
13	Summer2023!	11	3	68.5	9K years	Very Strong

Summary: 14 passwords audited, average entropy 56.4 bits, 6 weak, 6 strong.

Figure 7. HTML report (passwords masked, strength colour-coded)

The results illustrate the core insight of the project well: "password" and "123456" are flagged as Very Weak with crack-times under a second, while a 13-character mixed-class password like "7\$Kp#vL2!zQwR" is estimated at millions of years. Notably, "Welcome2024!" scores high on raw entropy but is still flagged for containing a dictionary word and a year - exactly the kind of weakness entropy alone would miss.

10. Testing Approach

The project uses JUnit 5 for unit testing, with every module covered by dedicated test classes. Tests follow the arrange-act-assert pattern and use JUnit's `@TempDir` for file-system tests, so no real files are touched. A total of 29 tests were written and all pass.

Test Class	Tests	Coverage
CharacterClassAnalyzerTest	3	Class detection, alphabet size, null safety
PatternDetectorTest	9	Every pattern type plus the clean-password case
EntropyCalculatorTest	3	Entropy math, class-diversity ordering
CrackTimeEstimatorTest	4	Crack-time scaling, human-readable formatting, validation
PasswordFileReaderTest	5	Trimming, missing files, binary-file guard, empty file
PasswordAnalyzerTest	5	End-to-end pipeline, immutability, null safety

Running the tests:

```
mvn test
```

The suite covers both the happy path and edge cases: null and empty inputs, overlong lines, non-positive attack rates, and immutability of the report object. Integration-style tests in `PasswordAnalyzerTest` exercise the full pipeline end to end, verifying that a weak password receives a low verdict and warnings while a strong one receives a high verdict with no patterns.

11. Challenges Faced

- Balancing entropy and pattern detection: early on, raw entropy gave "aaaaaaaaaa" a misleadingly high score. Adding the pattern detector alongside the math fixed this, but required careful design so the two results are both surfaced rather than merged into one number.
- Avoiding false positives in dictionary-word detection: short words like "in" or "or" appear inside almost any password. Restricting matches to words of length 4 or more reduced noise without losing meaningful detections.
- Keyboard-walk detection: a generic regex could not cleanly capture all QWERTY runs, so a sliding-window approach over the three keyboard rows (forward and reversed) was used instead.
- Masking passwords while keeping them recognisable: showing only the first and last character struck the right balance between privacy and usability.
- Keeping the build dependency-free: it was tempting to pull in a logging framework or a CLI parser, but each dependency would have widened the attack surface of a security tool. The standard library proved sufficient.
- HTML entity escaping: generating valid HTML from Java strings required a small escape helper to handle &, <, > and " correctly without breaking the generated page.

12. Learnings and Takeaways

- Entropy is necessary but not sufficient: a high entropy score does not mean a password is safe if it is built from predictable patterns. This is the central lesson of the project.
- Regex is powerful but easy to get subtly wrong: testing each pattern against known-good and known-bad inputs was essential to catching edge cases like reversed sequences.
- Separation of concerns pays off: because analysis, entropy and reporting are decoupled, adding the HTML reporter took only minutes and touched no analysis code.
- Immutability and the Builder pattern make reports safe to pass between reporters and trivially testable, with no defensive copying needed at call sites.
- Security tools should be minimal: every external dependency is a potential vulnerability or supply-chain risk. The zero-dependency constraint forced cleaner, more self-reliant code.
- Good error messages matter: handling missing files, null inputs and malformed lines with clear messages turned the tool from a toy into something genuinely usable.

13. Future Enhancements

- Adopt a zxcvbn-style probabilistic model that weights dictionary words, l33t substitutions and common patterns, giving a more realistic crack-time than the pure entropy formula.
- Ship larger bundled word lists (e.g. the Have I Been Pwned common-password corpus) and support loading them as gzip-compressed streams to keep the repo small.
- Add a simple GUI (JavaFX or a local web UI) with a per-character entropy heat-map for a more visual, interactive experience.
- Support password-manager CSV import (Bitwarden, 1Password) so users can audit their real vault without copy-pasting entries one by one.
- Add a JSON / CSV report output for programmatic consumption and integration into CI pipelines.
- Detect l33t-speak substitutions (a->@, e->3, i->1) and normalise before dictionary matching.
- Add per-password improvement suggestions (e.g. "add 2 more characters and a symbol to reach Strong").

14. References

- [1] Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27, 379-423.
- [2] Wheeler, D. (2017). zxcvbn: Low-Budget Password Strength Estimation. USENIX Security Symposium.
- [3] NIST Special Publication 800-63B: Digital Identity Guidelines - Authentication and Lifecycle Management.
- [4] Burr, W. E., Dodson, D. F., & Polk, W. T. (2013). NIST Electronic Authentication Guideline.
- [5] Oracle. Java 17 Documentation - java.util.regex and java.nio.file packages.
- [6] JUnit 5 User Guide: <https://junit.org/junit5/docs/current/user-guide/>
- [7] OWASP Password Storage Cheat Sheet and Authentication Cheat Sheet.
- [8] Have I Been Pwned - Pwned Passwords list: <https://haveibeenpwned.com/Passwords>